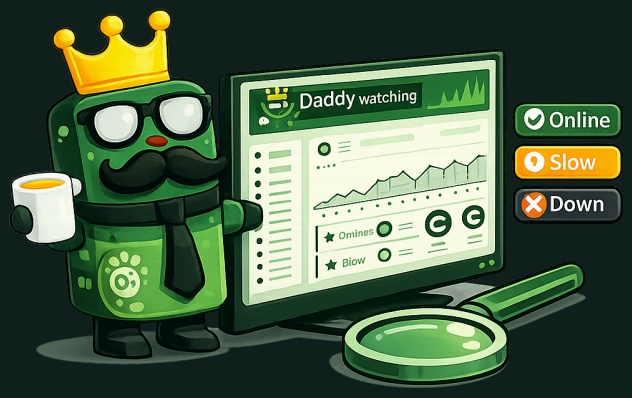
Uptime Daddy



Produkt Rapport

Uptime Daddy - Webbaseret Monitoreringssystem

Lavet af:

Daniel Steenberg

Kevin Nielsen

Kim Mortensen

Afleveringsdato:

D.16-04-2026

Indledning 4
Mål 4
Funktionelle krav 4
Ikke Funktionelle krav 4
Afgrænsninger 5
Vejledning: 5
Bruger oprettelse: 5
Tilføj din første website at monitorere: 6
Tjek en af de monitorerede website: 6
Produktdokumentation:
Kort beskrivelse af opbygning: 6
App (Frontend - Daniel - React): 7
Accounts (Backend - Daniel - Go): 8
Introduktion: 8
Golang: 8
Token: 11
Login: 12
Register: 12
ORM: 14
Serveropsætning(Server - Kim - Ubuntu): 14
Message broker (Mosquitto - Kim/Kevin): 15
DB opsætning (Backend - Kim - Postgress): 17
Raspberry Pi (IoT - Kim): 21
SSH tunnel og netværksadgang (Opsætning - Kim): 22
Automatisk genopkobling (autossh - Kim): 23
Systemd service (Opsætning - Kim): 23
Introduktion 25
Arkitektur: 25
Database: 26
Kommunikation (MQTT & Raspberry Pi) 26
Sikkerhed 27
Overvågning 27
Teknologivalg 27
API'ets opbygning i praksis 28
Håndtering af målinger 28

Kommunikation via MQTT 28
Håndtering af preview ping 29
Timeout og robusthed 30
WebsitesController og API-design 30
Designvalg 30
Fejlbehandling 31
HTTP Requester (Backend – Kim – Ruby) 31
Valg af programmeringssprog 31
Containerisering 32
Gems 33
Programstart 33
Bootloader 34
Hovedlogik (execute) 35
MQTT Broker 36
Kendt problem 41
Service 42
Tests 46
Mangler og videre arbejde 48
Konklusion 48

Kravspecifikation

Indledning

Uptime Daddy er et monitorerings værktøj, det ligger som en online website og bruges til at brugere kan vælge hvilke hjemmeside de gerne vil modtage data omkring, samt at se historisk data. Værktøjet skal kunne modtage websites og på en nem og grafisk måde give brugeren et overblik over statusen på de indtastede websites.

Mål

Målet med projektet har været at udvikle en bruger venlig platform som nemt og hurtigt giver et overblik over statuser på websites, det skal være brugeren der vælger disse websites hvorefter platformen skal levere grafer og stats.

Funktionelle krav

- Login og registering af bruger
- Bruger input af website til at få data fra
- Hente data fra websites samt logo
- Kontinuerlig data fra valgte sites
- Intern brug af message broker til at sende data fra IoT device og api
- Tokens til at verficere bruger
- Website stats med crud operationer
- Ruter skal være sikkerhedssikret gennem brug af JWT tokens
- Systemet skal understøtte refresh tokens til længerevarende session

Ikke Funktionelle krav

- Bruger venlig platform
- Ensarttet design
- Det må ikke være langsomt at modtage data fra valgte sites
- Page speed skal være i top og derfor indeks på de mange db entrys

- UI skal følge et konsistent farvetema og typografi
- Systemet skal kunne håndtere spikes i trafik uden nedbrud
- Systemet skal bruge asynkron behandling til tunge operationer
- Systemet må ikke blokere brugerinteraktion under dataindsamling
- Systemet skal være nemt at deploye
- Systemet skal være konfigurerbart via environment variables

Afgrænsninger

- CI/CD er ikke et krav fra starten
- Der er ikke modul basseret valg af grafer, der bliver som start kun give søjle og punkt diagrammer
- Kun manuelle password reset derved ingen glemt password funktionalitet
- Nogle sidder kan blokerer vores adgang dette godtages
- Det må ikke være langsomt at modtage data fra valgte sites
- Page speed skal være i top og derfor indeks på de mange db entrys
- Ingen bruger validering alle fake emails og random navne tilladt
- Der er ikke implementeret avanceret log-overvågning eller visualisering

Vejledning:

Bruger oprettelse:

1. Gå til website og her bliver du mødt af velkommen skærm click "new here etc..." (se bilag 1)

2. Du bliver nu mødt af en side med 3 input felter (se bilag 2) indtast navn, email og et password

3. Efter sucess bliver du sendt til at logge ind (bilag 3)

4. Indtast herefter din email samt password (se bilag 4)

5. Herefter bliver du sendt til dashboard (se bilag 5)

Tilføj din første website at monitorere:

1. I højre side af af skærmen under nav er der et input felt her indtaster du en url eks: dr.dk (se bilag 6)
2. Efter indtastning bliver der sendt en ping til den angivne hjemmeside og en modal åbner hvor du kan af/bekræfte om dette er korrekt samt få kort stats (se bilag 7)
3. Herefter vælger du en af de 2 options i bunden af denne modal (se bilag 8)
4. Du bliver sendt tilbage til dashboard hvor den valgte side nu er er i tabellen (se bilag 9)

Tjek en af de monitorerede website:

1. Du har en tabel på dashboarded der viser alle valte website at this point bør du kun have en enkelt klik på denne (se bilag 10)
2. Det åbner en modal hvor de data som er blevet samlet op bliver vist (se bilag 11)
3. I højre hjørne kan brugeren vælge hvor ofte der skal indsamles data by default hvert minut, tallet er opgjort i sekunder. (se bilag 12)
4. De forskellige stats er tilgængelige i toppen og har farve afhænigt af hvor hurtigt siden er, så grøn = godt, gul = iffy og rød er skidt (se bilag 13)
5. Under de kort med stats finder du første diagram her kan vælges i venstre del hvilken stat som skal vises i diagrammet samt hvor mange entries (se bilag 14)
6. I buden kan du vælge at slette det website du er inde på eller lukke modallen (se bilag 15)

Produktdokumentation:

Kort beskrivelse af opbygning:

Projektet er opbygget som en microservice arkitektur for at benytte specifikke sprog til specifikke problemmer og er helt generelt har vi en 3

delt opbygning nemlig: Frontend, backend og IoT. Vi vil nedenfor gennemgå hvilke teknologier der er brugt i disse kategorier.

Services:
App (frontend - Daniel - React)
Accounts (backend - Daniel - Go)
API (backend - Kevin - .net)
Server(Ubuntu - Kim)
IoT(Raspberry PI - Kim - Ruby)

App (Frontend - Daniel - React):

I frontenden andvender vi primært React men også et par filer med raw vanilla js, som css framework andvender vi semantic ui, til graferne bruger vi Recharts.

For at holde reacts bloat culture vedlige andvender vi som Atomic Design pattern som vores struktur i dette. Jeg vil nedenfor gennemgå hvorfor disse værktøjer er valgt.

React:
React er en af de største med brugte js framworks og fungere rigtigt godt, selv med dens fejl og mangler er det optimalt for os at bruge fordi vi gerne vil have muligheden for at bruge en komponent basseret approach og react state er nemme og gode at arbejde med hvilket giver en god dev oplevelse.

Semantic UI:
Det eneste der findes mere af end css frameworks er js framworks, men semantic ui er lightweight, nemt at bruge og giver en god mulighed for spice sine styling op med enten inline css eller deciderede css filer, samt levere nogle pæne og useable default komponenter som passede til vores stilart.

Recharts:
Valgt var mellem at selv at bygge nogle moduler der kunne vise de valgte diagrammet men med det time constraint vi havde gav det ikke mening at udvikle den dybe tallerken og recharts er userfriendly og havde de muligheder vi ledte efter og primært derfor faldt valget den vej.

Atomic Design pattern:
Som nævnt bliver react hurtigt fyldt med bloat men vores mål med frontenden var også at have flere komponenter som var reuseable og have en god mulighed for SoC, til dette er atomic design pattern rimelig optimalt da det udlistere ansvaret på en struktureret måde samt giver dokumentation for andre til at skulle overtage projektet så det ikke blot er en enkelt devs idér om struktur men giver en dokumentation fra hvor tingene bør findes samt laves.

Accounts (Backend - Daniel - Go):

Introduktion:

Vi har brugt en microservice arkitektur på platformen så derfor har account servicen ikke den helt store funktionalitet, det håndtere sådan set kun login, register og token udelingen. Dette er dog vigtige funktionalitet og er system kritisk da alle ruter til at modtage website data kræver en gyldig token.

Golang:

Vi har valgt at bruge Go fordi det er et light weight, hurtigt og effektivt sprog. Desuden tilbyder det en god udvikler oplevelse og er forholdvis readable for ikke stærke Go udviklere. Strukturen i Account servicen fungere således:
Go kræver en main.go fil i root for at kan køre, dog for at have bedre opdeling og struktur tilføjes blot en tom main.go, dette er for at

have et server lag hvor i alt til at starte servicen og kører servicen findes i cmd/server.

```
func main() {
    if err := godotenv.Load(".env"); err != nil {
        log.Fatal("Error loading .env file:", err)
    }
    port := os.Getenv("PORT")
    if err := db.Connect(); err != nil {
        log.Fatal("Failed to connect to DB:", err)
    }





    if err := db.DB.AutoMigrate(&models.Accounts{}); err != nil {
        log.Fatal("Failed to migrate DB:", err)
    }

    mux := http.NewServeMux()
    routes.RegisterAccountRoutes(mux, routes.AccountHandlers{
        Register: handler.CreateAccountHandler,
        Login:    handler.LoginHandler,
        Refresh:  handler.RefreshTokenHandler,
    })
    handlerWithCORS := middleware.WithCORS(mux)
    log.Printf("Server running on %s", port)
```

```
    if err := http.ListenAndServe(port, handlerWithCORS); err !=
nil {
        log.Fatal("Server failed:", err)
    }
}
```

Så i vores server.go loader vi vores env fil ind i projektet så de er tilgængelige derefter kaldes functionen til at connecte til vores postgres db, alt til at connecte til db ligger i mappen db.

Vores handlers til at styrer hvad der skal ske når x rute kaldes ligger i handler mappen pr rute ligger der en handler, så hvis noget går galt i forbindelse med en loginRequest skal problemmet debugges i loginHandler.go. Disse valg er taget med begrundelse i at nemme kunne debugge samt SoP in mind.

Vi definere vores ruter i routes mappen da vi primært nu har account relaterede ligger de alle sammen bare i en accounts.go, her kalder vi en mux funktion. Denne funktion matcher path med handler så når vi har

```
const (
    AccountsRegisterPath = "/accounts/register"
)
type AccountHandlers struct {
    Register http.HandlerFunc
}
func RegisterAccountRoutes(mux *http.ServeMux, handlers
AccountHandlers) {
    if handlers.Register != nil {
        mux.HandleFunc(AccountsRegisterPath,
postOnly(handlers.Register))
```

```
    }
```

Så f.eks i ovenstående definere vi ruten til at være /accounts/register, sammen med handler samt vælge hvilken operation skal tillades så i ovenstående tilfælde kan vi se at register er en post rute på stien /accounts/register som kalder Register handleren.

Efter vi i vores server startup har defineret ruter tilføjer vi CORS for at sørge for vores frontend rent faktisk har tilladelse til at sende request til vores service. Dette sker i middleware —> cors.go

Så det vigtige at forstå ift account servicen er at cmd/server binder tingene sammen og starter servicen op, handlers håndtere hvad der skal ske når en rute bliver kaldt, routes definere hvilke ruter der kan benyttes, models holder data modeller, db holder metoder til db.
Alt dette bliver startet op i docker med vores docker fil som bliver kørt detached for at det kører kontinuerligt på serveren.

Token:

Vi har valgt ikke at benytte private og public keys(asymetriske) for nu da vi blot bruger 2 services, så udviklingens tiden til at udvikle en auth service ville overstige det win der ville være ved at benytte auth servicen. Dog har vi valgt at lave både acessToken og refreshToken. Dette er med tanke i at brugeren kan have en random tab åben med monitorering, så ville det være en dårlig bruger oplevelse hvis de skulle logge ind og ud for at kunne benytte sig af servicen igen.

```
type Claims struct {

    UserID    uint   `json:"userId"`

    Email     string `json:"email"`

    FullName  string `json:"fullName"`

    TokenType string `json:"tokenType"`

    jwt.RegisteredClaims
```

```
}
```

Login:

Login fungere således at brugeren sender en request til accountservicen fra frontend som indholder et object der holder en email samt password. Servicen retunere så en signed acessToken og refreshToken, udover hvor disse expire samt en status besked.

```
type LoginRequest struct {
    Email    string `json:"email"`
    Password string `json:"password"`
}
type LoginResponse struct {
    Message          string `json:"message"`
    AccessToken      string `json:"accessToken"`
    RefreshToken     string `json:"refreshToken"`
    AccessExpiresAt  int64  `json:"accessExpiresAt"`
    RefreshExpiresAt int64  `json:"refreshExpiresAt"`
}
```

Register:

En bruger skal bruge en login til at benytte platformen for at logge ind men også for at få adgang til servicen, da som tidligere nævn kræves der token på alle ruter til backend.

Når en bruger laver et register kald bliver der kaldt på ruten:
`AccountsRegisterPath = "/accounts/register"`

Som så bliver matchet med register handleren, register handleren modtager et object med nedenstående:

```
  type CreateAccountRequest struct {
    Email    string `json:"email"`
    FullName string `json:"fullName"`
    Password string `json:"password"`
}
```

Så bruger vi bcrypt til at hashe passworded så vi ikke gemmer det "rigtige" password direkte i db så hvis der var et leak ville password stå i plaintext som så kunne udnyttes. Vi laver derefter en account i db med disse:

```
    account := models.Accounts{
        Email:    req.Email,
        Password: string(hashedPassword),
        FullName: req.FullName,
    }
```

Så vi pr register request får en en account som matcher vores schema:

```
type Accounts struct {
   gorm.Model
```

```
    Email    string `gorm:"uniqueIndex;not null"`

    Password string `gorm:"column:password_hash;not null"`

    FullName string `gorm:"column:fullName;not null"`

}
```

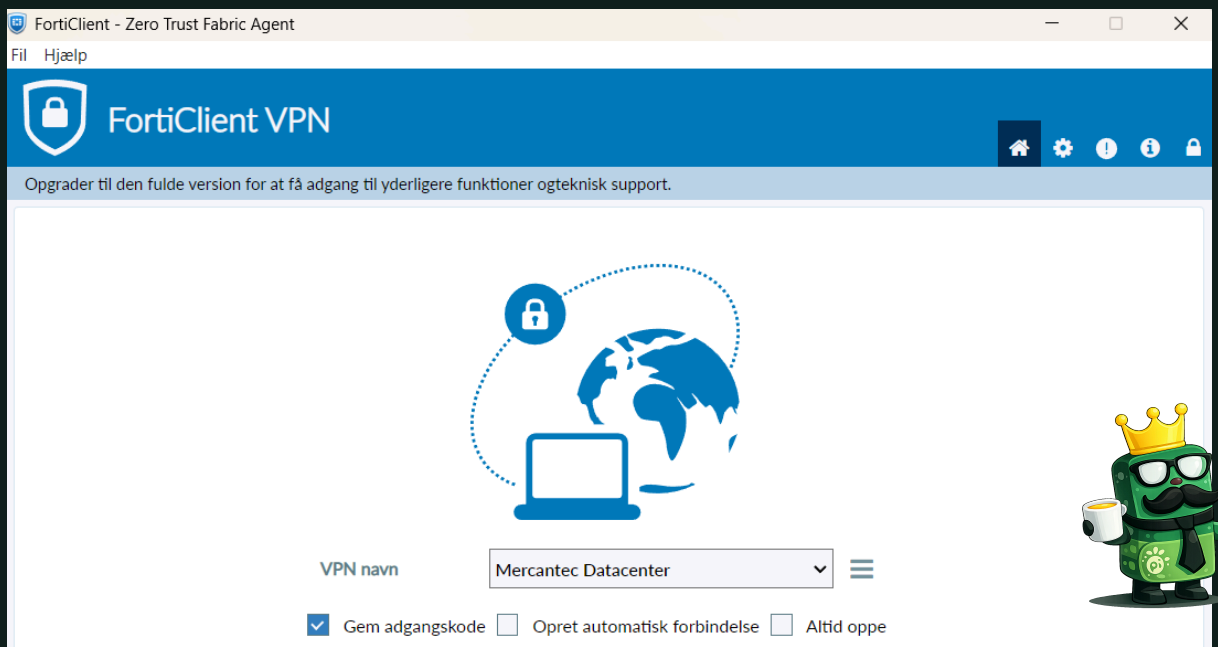
ORM:

Som man kan se ovenstående har vi brugt gorm som ORM, vi har valgt at bruge Gorm for at slippe for det store mængde boilerplate som der ville være ved at “hardcode” sql, samt det er nemt og bekvemt at benytte sig af.

Serveropsætning(Server - Kim - Ubuntu):

I dette projekt har vi valgt at benytte en server stillet til rådighed af skolen, da dette sikrer os en gratis server, som altid er tilgængelig. Vi har valgt at anvende en Ubuntu Server, da den er nem og hurtig at opsætte til de fleste behov, og samtidig er det den Linux-distribution, vi har mest erfaring med.

For at få adgang til serveren anbefaler skolen brugen af FortiClient VPN, hvilket vi derfor har anvendt som en del af den indledende opsætning.



Når serveren er oppe at køre, er det første vi gør at installere Docker og Docker Compose. Dette gør det muligt at køre applikationer i containere, hvilket gør systemet nemmere at vedligeholde og deploye.

```
sudo apt update
sudo apt install docker.io docker-compose -y
```

Derefter opsætter vi en Docker-container, som kører en Twingate-klient. Twingate bruges til at skabe en sikker forbindelse til serverens netværk uden at skulle åbne porte direkte til internettet.

Dette betyder, at alle gruppemedlemmer kan få adgang til serveren via Twingate, uden at skulle konfigurere VPN individuelt eller eksponere serveren offentligt. Det øger både sikkerheden og gør opsætningen nemmere at arbejde med i gruppen.

Herefter oprettede vi en bruger til hver af os, så vi alle kunne logge ind med vores egne brugere. Vi har valgt, at kun Kim har sudo-rettigheder, hvilket skaber en klar ansvarsfordeling og øger sikkerheden på serveren.

```
sudo adduser krixzy
sudo usermod -aG sudo krixzy
```

Til sidst blev alle brugere tilføjet til Docker-gruppen, så de kan køre Docker-kommandoer uden at skulle bruge sudo.

```
sudo usermod -aG docker username
```

Efter dette var serveren fuldt opsat og klar til brug i projektet.

Message broker (Mosquitto - Kim/Kevin):

Efter serveren var opsat og tilgængelig for alle gruppemedlemmer, installerede vi Mosquitto, som vi har valgt at bruge som vores message broker.

Vi valgte at køre Mosquitto som en service direkte på serveren i stedet for i Docker, selvom resten af systemet kører i containere. Dette skyldes, at Mosquitto fungerer som en central infrastrukturkomponent. Den skal være stabil, starte automatisk sammen med serveren og være uafhængig af resten af container-opsætningen. Derfor vurderede vi, at en systemservice var den mest enkle og robuste løsning.

```
sudo apt install mosquitto mosquitto-clients -y
sudo systemctl enable mosquitto
sudo systemctl start mosquitto
```

Mosquitto er en message broker. En message broker er en service, som håndterer kommunikation mellem forskellige enheder via MQTT.

MQTT er en kommunikationsprotokol, der bruges til at sende små beskeder mellem enheder over et netværk. Den er særligt velegnet til IoT-enheder, hvor forbindelsen kan være langsom eller ustabil.

Grundlæggende fungerer MQTT sådan, at i stedet for at sende data direkte mellem enheder (som man gør med f.eks. HTTP), kan en eller flere enheder abonnere på et såkaldt *topic*.

```
mosquitto_sub -t "test_besked"
```

Dette kan gøres fra flere programmer, som har adgang til samme netværk og kan kommunikere med Mosquitto-serveren.

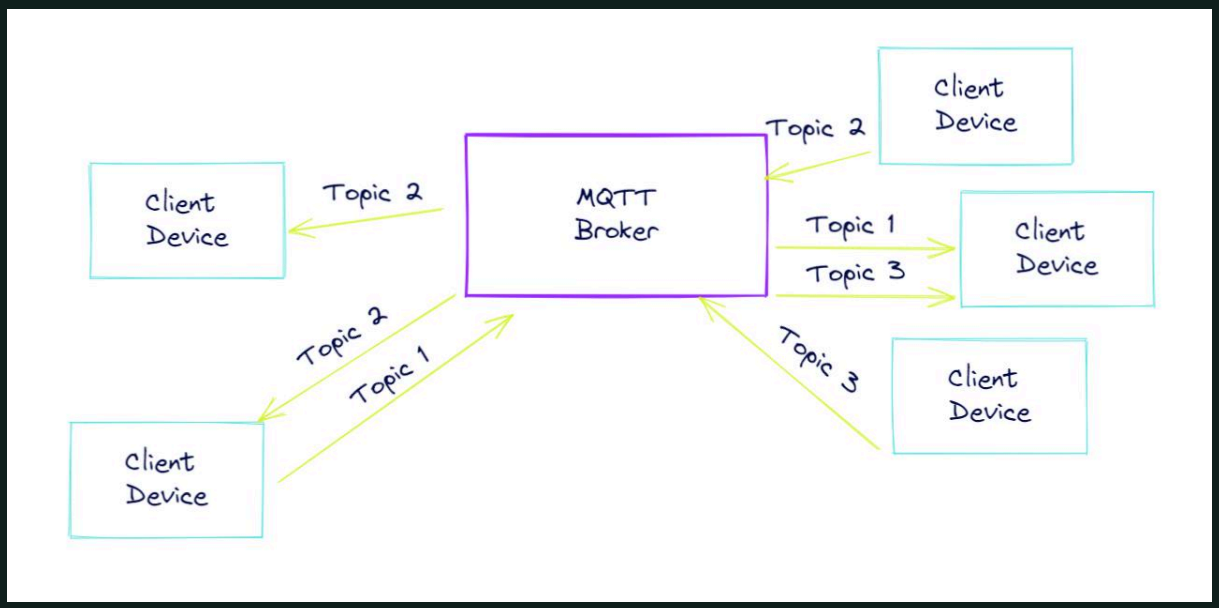
Hvis man derefter sender en besked til det samme topic:

```
mosquitto_pub -t "test_besked" -m "dette er beskeden"
```

vil alle enheder, der abonnerer på dette topic, modtage beskeden.

Mosquitto understøtter også funktioner som lagring af beskeder, hvilket betyder, at hvis en enhed midlertidigt mister forbindelsen, kan den modtage de beskeder, den har misset, når forbindelsen genoprettes. Dette er særligt

nyttigt i IoT-systemer, hvor forbindelsen ikke altid er stabil.



DB opsætning (Backend - Kim - Postgress):

Til vores databaseopsætning har vi oprettet en Docker Compose-fil, som starter både en PostgreSQL-database og en pgAdmin-container.

PostgreSQL er den database, vi har valgt at anvende (se mere i databaseafsnittet), mens pgAdmin giver os et webbaseret interface, som gør det nemt at administrere databasen.

```
services:
  db:
    image: postgres:16
    container_name: postgres-db
    restart: unless-stopped
    env_file:
      - /shared/database/.env
    ports:
      - "5432:5432"
```

```yaml
    volumes:
      - postgres_data:/var/lib/postgresql/data


  pgadmin:
    image: dpage/pgadmin4
    container_name: pgadmin
    restart: unless-stopped
    env_file:
      - /shared/database/.env
    ports:
      - "5050:80"
    volumes:
      - pgadmin_data:/var/lib/pgadmin


volumes:
  postgres_data:
  pgadmin_data:
```

For at håndtere fælles data på serveren har vi oprettet en mappe på serveren:

```
/shared/database
```

Denne mappe bruges til at dele database-relaterede filer og konfiguration mellem gruppemedlemmer og containere.

Vi har samtidig oprettet en brugergruppe kaldet *database_admin*, som har adgang til denne mappe. Gruppen har fået læse-, skrive- og execute-rettigheder, så alle medlemmer kan arbejde med filerne.

```
sudo mkdir -p /shared/database
sudo chown :database_admin /shared/database
sudo chmod 770 /shared/database
Note: Kommandoen chmod 770 giver ejeren af mappen læse-, skrive- og
execute-rettigheder. Det første 7-tal gælder ejeren, det andet 7-tal
gælder gruppen, mens det sidste 0 betyder, at alle andre brugere ikke
har nogen adgang.
Tallet 7 repræsenterer summen af read (4), write (2) og execute (1),
hvilket giver fulde rettigheder.
```

Dette sikrer, at kun brugere i gruppen *database_admin* har adgang til mappen, hvilket både gør samarbejdet nemmere og øger sikkerheden

SSH-forbindelse (Opsætning - Kim):

For at tilgå serveren bruger vi SSH (Secure Shell). SSH er en protokol, der bruges til at oprette en fjernforbindelse til en server. Forbindelsen er krypteret, hvilket gør det væsentligt mere sikkert end ældre løsninger som f.eks. Telnet.

For at oprette forbindelse via SSH kan man bruge følgende kommando:

```
ssh username@server-ip
```

En bedre og mere sikker løsning er at bruge SSH-nøgler i stedet for password. Derfor har vi valgt at opsætte en SSH-konfiguration med nøglebaseret login.

Først genererer man et nøglepar på sin egen computer:

```
ssh-keygen -t rsa -b 4096
```

Herefter går man ind på serveren og opretter en .ssh mappe i sin hjemmemappe, hvis den ikke allerede findes. Inde i denne mappe oprettes en fil kaldet authorized_keys.

I denne fil indsætter man indholdet af sin offentlige nøgle (.pub-filen). Dette gør, at serveren kan genkende brugeren, og man kan logge ind uden at skulle bruge password.

På sin egen computer kan man derefter oprette eller redigere SSH-konfigurationsfilen i .ssh/config.

```
Host pi
  HostName 192.168.0.85
  User krixzy

Host skole
  HostName 10.133.51.156
  User krixzy
  IdentityFile ~/.ssh/id_rsa

Host H5
  HostName 10.133.51.121
  User krixzy
  IdentityFile ~/.ssh/id_rsa
```

- Host er et valgfrit navn for forbindelsen
- HostName er serverens IP-adresse
- User er den bruger, man logger ind som
- IdentityFile bruges til at specificere hvilken SSH-nøgle der anvendes til forbindelsen, hvilket gør det muligt at logge ind uden password.

Hvis IdentityFile ikke er angivet, vil SSH automatisk forsøge at bruge standardnøgler som ~/.ssh/id_rsa.
Når dette er sat op, kan man oprette forbindelse ved blot at skrive:

```
ssh H5
```

Så er du inde på serveren.

Raspberry Pi (IoT - Kim):

Raspberry Pi'en fungerer som vores IoT-enhed i projektet og bruges til at udføre løbende HTTP-requests mod eksterne hjemmesider.

Enheden tilgås via SSH med nøglebaseret login, som beskrevet i det tidligere afsnit.

Vi har valgt en Raspberry Pi, da den har mere regnekraft end mange traditionelle IoT-enheder, samtidig med at den har et lavt strømforbrug og er nem at arbejde med. Enheden kører en let version af Ubuntu, hvilket gør det nemt at installere og konfigurere software.

Selvom denne opgave teknisk set kunne være udført direkte på serveren, har vi valgt at anvende en Raspberry Pi for at skabe en bedre opdeling af ansvar. Ved at lade Raspberry Pi'en håndtere de gentagne requests, undgår vi at belaste serveren unødigt.

Dette betyder også, at selv hvis serveren er ustabil eller midlertidigt utilgængelig, vil Raspberry Pi'en fortsat kunne udføre HTTP-requests kontinuerligt og dermed sikre en mere stabil dataindsamling.

På enheden har vi opsat to brugere:

En personlig bruger til administration og udvikling

En *app runner*-bruger, som udelukkende bruges til at køre applikationen i produktionsmiljø

SSH tunnel og netværksadgang (Opsætning - Kim):

Da Raspberry Pi'en ikke selv har adgang til skolens VPN, har vi opsat en SSH-tunnel, så den kan tilgå serveren via en mellemstation.

Løsningen fungerer ved, at Raspberry Pi'en først forbinder til en lokal computer, som har adgang til netværket, og derefter videre til skolens server via Twingate.

Dette er konfigureret via en SSH config-fil:

```
Host LOCAL
HostName 192.168.0.129
User morte

Host H5
HostName 10.133.51.121
User krixzy
ProxyJump LOCAL
LocalForward 0.0.0.0:1883 localhost:1883
LocalForward 0.0.0.0:8080 localhost:8080
LocalForward 0.0.0.0:6969 localhost:6969
ServerAliveInterval 10
ServerAliveCountMax 3
ExitOnForwardFailure yes
BatchMode yes
```

- ProxyJump bruges til at hoppe via en mellemstation
- LocalForward videresender porte fra serveren til Raspberry Pi'en
- ServerAliveInterval holder forbindelsen aktiv pinger hvis der ikke har været kontakt i 10 sec
- ServerAliveCountMax hvor mange gange til vel prøve at reconnect ved tabt forbindelse
- ExitOnForwardFailure stopper forbindelsen hvis port forwarding fejler
- BatchMode sikrer, at der ikke forventes brugerinput (fx password) og fejler hvis der bliver spurt om det

De forwarded porte bruges til følgende:

- 1883 → MQTT (Mosquitto)
- 8080 → Backend API
- 6969 → Account service (bruges til autentificering)

Vi har valgt at forwarde portene til `0.0.0.0` i stedet for `localhost`, da vores Docker-containere ikke kører på samme netværk og derfor ikke kan tilgå `localhost` direkte.

Ved at bruge `0.0.0.0` bindes portene til alle netværksinterfaces, hvilket gør det muligt for eksterne forbindelser – herunder vores containere – at tilgå de forwarded porte.

Dette sikrer, at de forskellige dele af systemet kan kommunikere, selvom de ikke er på samme interne netværk

Automatisk genopkobling (autossh - Kim):

Da forbindelsen til skolens server kan være ustabil, har vi anvendt autossh til automatisk at genoprette forbindelsen.

```
autossh -M 0 -N H5
```

- -M 0 deaktiverer ekstra monitor-port (mere stabil opsætning)
- -N betyder, at der ikke åbnes en terminal
- H5 refererer til konfigurationen i SSH config-filen

Systemd service (Opsætning - Kim):

For at sikre, at forbindelsen altid er aktiv, har vi opsat autossh som en systemd service:

```
[Unit]
Description=Persistent SSH tunnel to H5 through LOCAL
After=network-online.target
Wants=network-online.target

[Service]
User=krixzy
Environment="AUTOSSH_GATETIME=0"
ExecStart=/usr/bin/autossh -M 0 -N H5
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

- After siger at denne service ikke kan starte før network køre
- Wants siger at network service skal startes fordi denne service skal bruge
- User angiver hvilken bruger der kører servicen
- Environment="AUTOSSH_GATETIME=0" gør, at autossh starter uden forsinkelse
- ExecStart er den kommando, der køres
- Restart=always sørger for automatisk genstart
- RestartSec angiver ventetid før genstart
- WantedBy=multi-user.target betyder, at servicen starter automatisk ved boot multi-user.target er bare det stadie systemet skal være i før den må starte

Servicen kan startes manuelt her og nu med kommandoen:

```
systemctl start h5-tunnel
```

Da servicefilen hedder h5-tunnel, er det også dette navn, der bruges til at starte servicen.

Man kan også aktivere servicen ved boot med:

```
systemctl enable h5-tunnel
```

Dette gør, at servicen automatisk startes igen, når systemet genstartes.

API (Backend - Kevin - .NET):

Introduktion

I dette projekt har jeg arbejdet med udviklingen af API-backenden, som er implementeret i C# ved brug af ASP.NET Core (.NET 10). Backendens formål er at håndtere data relateret til websites samt stille et REST API til rådighed, som frontend-applikationen kan kommunikere med.

Backenden fungerer som bindeled mellem databasen og systemets worker-enheder, som består af Raspberry Pi, der udfører målinger på de overvågede websites.

Arkitektur:

Backend-projektet er struktureret efter en klassisk ASP.NET Core arkitektur, hvor koden er opdelt i mapper med klart definerede ansvarsområder. Dette gør systemet mere overskueligt og nemmere at vedligeholde.

Projektet indeholder blandt andet:

- Controllers – håndterer HTTP requests og returnerer responses
- Models – repræsenterer datastrukturen i systemet
- DTOs – bruges til dataoverførsel mellem frontend og backend
- Services – indeholder forretningslogikken
- Data – håndterer databaseforbindelsen
- Migrations – bruges til versionsstyring af databasen
- HealthChecks – overvåger status på API, database og MQTT
- UnitTests – tester backend-logikken

Applikationens entry point er *Program.cs*, hvor services registreres, middleware konfigureres, og API'et startes.

Projektet anvender Docker til containerisering, hvilket sikrer et ensartet miljø og gør deployment nemmere.

Database:

Databasen er opbygget i PostgreSQL og tilgås via Entity Framework Core.

Systemet består af tre centrale tabeller:

- Accounts
- Websites
- Measurements

Relationerne mellem tabellerne er:

- Én bruger kan have mange websites
- Ét website kan have mange measurements
- Hver measurement tilhører ét website

Denne struktur kan beskrives som:

Account → Website → Measurement

Measurements indeholder blandt andet data som svartider (DNS lookup, connection, TLS, time to first byte og total tid), hvilket gør det muligt at analysere performance og oppetid.

Kommunikation (MQTT & Raspberry Pi)

En central del af backendens funktionalitet er kommunikationen med systemets worker-enheder.

Kommunikationen foregår via MQTT, hvor backend sender forespørgsler om målinger til Raspberry Pi-enhederne. Disse enheder udfører målingerne og sender resultaterne tilbage til backend, som gemmer dem i databasen.

Denne asynkrone kommunikation gør systemet fleksibelt og skalerbart, da flere worker-enheder kan tilføjes uden ændringer i backendens struktur.

En mere detaljeret gennemgang af MQTT-implementeringen findes i afsnittet nedenfor.

Sikkerhed

For at beskytte API'et anvendes JWT (JSON Web Tokens). Dette betyder, at brugere skal være autentificerede for at kunne tilgå systemets endpoints.

Token-validering er konfigureret i backendens opsætning og sikrer, at kun autoriserede brugere kan hente eller ændre data.

Overvågning

Backend-systemet overvåges ved hjælp af healthchecks og Prometheus.

Healthchecks bruges til at verificere status på:

- API'et
- Databasen
- MQTT-forbindelsen

Dette gør det muligt hurtigt at identificere fejl i systemet.

Teknologivalg

ASP.NET Core er valgt på grund af høj performance og god understøttelse af REST API'er.

Entity Framework Core gør det nemt at arbejde med databasen gennem objekter.

MQTT er valgt som kommunikationsprotokol, da det understøtter letvægts og asynkron kommunikation, hvilket er ideelt til IoT-enheder.

Docker er anvendt for at sikre ensartet opsætning og nem deployment.

API'ets opbygning i praksis

I backend-implementeringen er API'et bygget op efter en lagdelt struktur, hvor hver del af systemet har et tydeligt ansvar.

Når frontend sender en request til et endpoint, bliver denne først håndteret i en controller. Controllerens ansvar er at modtage input og sende det videre til service-laget.

Service-laget indeholder forretningslogikken og står for behandling af data, herunder oprettelse og lagring i databasen.

For at undgå at eksponere databasedata direkte til frontend anvendes DTO'er, som kontrollerer hvilke data der sendes mellem systemets lag.

Håndtering af målinger

Når en Raspberry Pi udfører en måling på et website, bliver resultatet sendt tilbage til API'et via MQTT.

Backenden modtager disse data, omsætter dem til interne objekter og gemmer dem i databasen som measurements. Hver measurement er knyttet til et specifikt website, hvilket gør det muligt at vise historiske data og statistik i frontend.

Denne struktur gør det muligt at opsamle store mængder data uden at ændre systemets opbygning.

Kommunikation via MQTT

MQTT-kommunikationen er implementeret i to retninger: publish og subscribe.

`MqttPublishService` bruges til at sende beskeder til worker-enhederne. Beskederne sendes som JSON til specifikke topics, eksempelvis:

- uptime/websites/created
- uptime/websites/deleted
- uptime/websites/updated
- uptime/ping/requests

`MqttService` fungerer som en background service, der lytter på MQTT-topics.

```
await _client.SubscribeAsync("uptime/measurements");
```

Dette gør det muligt for backend løbende at modtage målinger fra worker-enhederne.

Når data modtages, bliver de deserialiseret og gemt i databasen efter validering.

Håndtering af preview ping

`PingPreviewService` bruges til at udføre preview-målinger, før et website oprettes i systemet.

Når en bruger sender en preview-request, genereres et unikt requestId, som bruges til at matche svaret fra Raspberry Pi.

```
_pendingRequests.TryAdd(requestId, tcs);
```

Her gemmes requesten i en `ConcurrentDictionary`, så flere forespørgsler kan håndteres parallelt.

```
return await tcs.Task;
```

Denne linje gør, at API'et venter på et svar fra MQTT, så frontend oplever det som en almindelig synkron request.

Timeout og robusthed

For at sikre robusthed er der implementeret timeout på preview-requests.

Hvis der ikke modtages svar inden for 10 sekunder, annulleres forespørgslen automatisk, og den fjernes fra systemet.

Dette forhindrer memory leaks og sikrer stabil drift.

WebsitesController og API-design

`WebsitesController` håndterer alle endpoints relateret til websites.

Controlleren er beskyttet med `[Authorize]`, og brugerens id udtrækkes fra JWT-tokenet:

```
var userIdClaim = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
```

Dette sikrer, at backend selv bestemmer, hvilken bruger data tilhører.

Controlleren indeholder endpoints til:

- Hentning af websites
- Hentning af websites med measurements
- Oprettelse af websites
- Sletning af websites
- Opdatering af interval

Ved oprettelse valideres input, og det kontrolleres blandt andet, at websitet ikke allerede eksisterer.

Designvalg

Et vigtigt designvalg er brugen af projektion (`Select(...)`) ved datahentning.

Dette betyder, at kun nødvendige felter sendes til frontend, hvilket:

- reducerer datamængden
- forbedrer performance
- øger sikkerheden

Derudover sorteres measurements efter `CreatedAt`, så de nyeste data vises først.

Fejlbehandling

API'et indeholder tydelig fejlhåndtering.

Hvis input er ugyldigt, returneres meningsfulde fejlbeskeder. Hvis data ikke findes, returneres `NotFound`.

I preview ping håndteres timeout med statuskode 408.

I MQTT-servicen logges fejl og ugyldige payloads, så systemet ikke bryder sammen ved uventede data.

HTTP Requester (Backend – Kim – Ruby)

HTTP Requester er det program, som kører på vores Raspberry Pi. Programmet henter en liste af hjemmesider via et API og udfører løbende HTTP-requests på disse for at overvåge deres tilgængelighed og svartid.

Resultaterne sendes derefter tilbage til backend, hvor de gemmes og anvendes i systemet.

Kommunikationen mellem Raspberry Pi'en og backend sker via en message broker, hvor vi anvender Mosquitto, som er hostet på skolens server.

Valg af programmeringssprog

Ruby er valgt som programmeringssprog til HTTP Requester, da det har en læsbar syntaks og høj udviklingshastighed, hvilket gør det velegnet til

automatisering.

Da programmet primært arbejder med netværkskommunikation (HTTP-requests og MQTT) og ikke er beregningstungt, er Ruby velegnet til opgaven. Programmet er I/O-baseret, hvilket betyder, at det meste af tiden bruges på at vente på svar fra eksterne systemer frem for at udføre tunge beregninger.

Samtidig har Ruby gode biblioteker (gems), som gør det nemt at implementere funktionaliteten.

Derudover har vi valgt Ruby, da det er et sprog, vi har erfaring med. Dette har gjort det muligt at udvikle løsningen hurtigere og med færre fejl.

Containerisering

HTTP Requester er bygget til at køre i en Docker-container. Docker sikrer, at applikationen kører i et ensartet miljø uanset værtsmaskinen, og gør det nemmere at deploye løsningen, da hele miljøet er samlet i ét image.

Projektet er struktureret med en klar opdeling:

- **Root** indeholder bl.a. docker-compose.yml og README.md
- **http_requester/** indeholder selve applikationen
  - main.rb
  - Gemfile
  - Dockerfile
  - lib/ → selve programlogikken
  - spec/ → tests

Mappen lib/ indeholder

- en bootloader
- http_requester kassen
- service-klasser
- model-klasser

Mappen spec/ bruges til unit tests.

Gems

Vi anvender følgende gems:

```
gem "faraday"
gem "faraday-retry"
gem "faraday-follow_redirects"
gem "mqtt"
gem "dotenv"
gem "rspec"
```

- **Faraday** → bruges til HTTP-requests
- **faraday-retry** → gør det muligt at retry requests
- **faraday-follow_redirects** → håndterer redirects (fx 301)
- **mqtt** → bruges til kommunikation med message broker
- **dotenv** → bruges til environment variables
- **rspec** → bruges til tests

Gems er eksterne pakker, som gør det nemmere at udvikle funktionalitet.

Programstart

Programmet startes via `main.rb`:

```
require_relative "lib/boot_loader"

@logger = Logger.new($stdout)
@http_requester = HttpRequester.new(@logger)
@http_requester.execute
```

Her:

- loader vi alle dependencies ved at loade bootloader
- opretter en logger som er en flotter måde at lave stdout på med timestamp og så kan den vise om det er error beskeder eller bare info som set her

```
I, [2026-04-15T09:49:50.933624 #1]  INFO -- : Calculating unique
pages needing ping...
I, [2026-04-15T09:49:55.933873 #1]  INFO -- : Calculating unique
pages needing ping...
I, [2026-04-15T09:49:55.934043 #1]  INFO -- : Calculating unique
pages needing ping...
I, [2026-04-15T09:49:55.934819 #1]  INFO -- : Pinging https://dr.dk
for 1 page(s)
I, [2026-04-15T09:49:56.197472 #1]  INFO -- : Received response for
https://dr.dk: {"status"=>"200", "dns_lookup"=>27.03,
"connect_to_page"=>48.05, "tls_hand_shake"=>113.08,
"time_to_first_byte"=>146.15, "total_time"=>236.84}
I, [2026-04-15T09:49:56.197598 #1]  INFO -- : Calculating unique
pages needing ping...
I, [2026-04-15T09:49:56.197687 #1]  INFO -- : Queueing MQTT publish
to 'uptime/measurements'
I, [2026-04-15T09:49:56.479157 #1]  INFO -- : Publishing MQTT message
to 'uptime/measurements':
{"pages":[{"id":4,"path":"dr.dk","user_id":3,"response":{"status":"20
0","dns_lookup":27.03,"connect_to_page":48.05,"tls_hand_shake":113.08
,"time_to_first_byte":146.15,"total_time":236.84}}]}
```

- initialiserer HTTP Requester
- starter programmet

Bootloader

Bootloaderen sørger for at indlæse alle nødvendige dependencies:

```
require "faraday"
require "faraday/retry"
require 'json'
require 'logger'
require "mqtt"
require "thread"
require "open3"
require "dotenv"
```

```ruby
require "faraday/follow_redirects"
require "base64"

Dotenv.load(File.expand_path("../../.env", __dir__))

Dir[File.join(__dir__, "**/*.rb")].sort.each do |file|
  next if file == __FILE__
  require file
end
```

Den:

- loader gems
- loader environment variables
- loader alle interne filer automatisk

Vi har valgt at anvende en bootloader til at håndtere indlæsning af dependencies og interne filer i stedet for at gøre det direkte i main.rb. Dette samler alle dependencies ét sted, hvilket gør koden nemmere at vedligeholde og giver et bedre overblik over projektets opbygning.

En ulempe ved denne tilgang er, at der kan blive indlæst flere filer end nødvendigt, da alle filer i projektet automatisk bliver loaded. I dette projekt har det dog ikke nogen væsentlig betydning, da applikationen er relativt lille, og overheaden derfor er minimal.

Hovedlogik (execute)

Den centrale logik ligger i execute metoden:

```ruby
def execute
  start_immediate_ping_worker!
  start_mqtt_client!
  load_websites_from_api!
  fetch_missing_favicons!

  until stop
```

```
    begin
      update_page_handler! if state.changes?
      run_curl_on_pages! if page_handler.pages_needing_ping?
    rescue => e
      logger.error("An error occurred in event loop: #{e.message} \n
#{e.backtrace.join("\n")}")
    end
    sleep(5)
  end
end
```

Først startes en *Immediate Ping Worker*, som konstant lytter efter såkaldte *quick pings* fra Mosquitto.

Quick pings bruges af frontenden til at validere en hjemmeside, når en bruger indtaster en URL. Her sendes en forespørgsel til Raspberry Pi'en, som hurtigt laver et ping på siden og returnerer resultatet, så brugeren kan få feedback med det samme.

Herefter startes en MQTT-klient, som lytter efter beskeder fra vores Mosquitto message broker. Disse beskeder kan f.eks. indeholde nye hjemmesider, opdateringer eller sletninger.

Derefter hentes alle eksisterende hjemmesider fra backend via et API, så Raspberry Pi'en har et opdateret udgangspunkt.

Efterfølgende gennemgås listen af hjemmesider for at se, om nogle mangler favicons. Hvis dette er tilfældet, hentes de automatisk.

Herefter starter programmets main loop, som kører kontinuerligt:

- Først tjekkes det, om der er kommet nye ændringer fra MQTT
- Hvis der er, opdateres systemets state og page_handler
- Derefter tjekkes det, om der er hjemmesider, der skal pinges
- Hvis dette er tilfældet, sendes HTTP-requests, og resultaterne sendes videre via MQTT

Vores MQTT broker er implementeret som en klasse, der håndterer både modtagelse og afsendelse af beskeder.

Den er opbygget således, at der startes en subscriber og en publisher i hver sin thread. Dette gør det muligt at håndtere indgående og udgående beskeder uafhængigt af hinanden.

```
def start_subscriber!
    Thread.new do
      loop do
        begin
          logger.info("Connecting subscriber to MQTT broker at
#{mqtt_host}:1883...")
          MQTT::Client.connect(
            host: mqtt_host,
            port: ENV.fetch("MQTT_PORT", 1883),
            keep_alive: 5
          ) do |client|
            client.subscribe(create_page_topic, delete_page_topic,
ping_request_topic, update_page_topic)
            logger.info("Connected subscriber to MQTT broker")
            client.get do |topic, message|
              logger.info("Received message on '#{topic}': #{message}")
              handle_response(topic, message)
            end
          end
        rescue MQTT::ProtocolException, Errno::ECONNREFUSED, IOError,
SystemCallError => e
          logger.error("Subscriber connection error: #{e.class} -
#{e.message}")
          sleep 10
        rescue => e
          logger.error("Subscriber unexpected error: #{e.class} -
#{e.message}\n#{e.backtrace.join("\n")}")
          sleep 10
        end
      end
    end
  end
```

Vi har valgt at anvende threads, da systemet har flere opgaver, som skal køre samtidig og uafhængigt af hinanden.

HTTP Requester består både af et main event loop, som kører hvert 5. sekund, og en MQTT-komponent, som skal kunne modtage beskeder i realtid. Hvis MQTT-håndteringen blev placeret i main loopet, ville systemet kun lytte efter beskeder med intervaller, hvilket ville skabe forsinkelse og gøre systemet mindre responsivt.

Ved at køre subscriber og publisher i separate threads kan MQTT-klienten lytte kontinuerligt, uafhængigt af hvad main loopet laver. Dette er nødvendigt, da MQTT's client.get er blokerende og ellers ville stoppe resten af programmet.

Threads gør det derfor muligt at opdele systemet i uafhængige komponenter:

- Main loop håndterer planlagte HTTP-requests
- Subscriber håndterer indgående beskeder i realtid
- Publisher håndterer afsendelse af data

Denne opdeling sikrer, at systemet både kan reagere hurtigt på ændringer og samtidig udføre løbende arbejde uden at blokere.

Threads i Ruby kører inden for samme proces og deler samme CPU, hvilket betyder, at de ikke nødvendigvis kører parallelt, men i stedet skiftes til at bruge CPU'en. Dette styres af Ruby's interpreter og gør, at flere opgaver kan fremstå som samtidige.

Når der modtages beskeder fra MQTT, bliver de ikke behandlet direkte, den sortere kun beskderne der kommer ind og sender dem til en state-model, som main loopet senere håndterer.

```
class State
```

```ruby
  attr_reader :logger, :mutex
  attr_accessor :changes

  def initialize(logger)
    @logger = logger
    @changes = []
    @mutex = Mutex.new
  end

  def changes?
    mutex.synchronize do
      !changes.empty?
    end
  end

  def add_change(change)
    mutex.synchronize do
      logger.info("Adding change: #{change.to_s}")
      self.changes << change
    end
  end

  def fetch_changes
    mutex.synchronize do
      current_changes = changes.dup
      changes.clear
      current_changes
    end
  end

  private

    def clean_changes!
      mutex.synchronize do
        logger.info("Clearing changes: #{changes.map(&:to_s).join(', ')}")
        changes.clear
      end
    end
end
```

I state-klassen anvendes en Mutex til at sikre, at flere threads ikke ændrer data samtidig.

Ved at bruge:

```
mutex.synchronize do
```

åses adgangen til data, så kun én thread kan arbejde med den ad gangen. Dette gør løsningen thread-safe og forhindrer race conditions.

vi har valgt at anvende en state i stedet for en queue til at håndtere indgående ændringer.

En queue bruges typisk til at behandle data én ad gangen, mens vores behov er at samle ændringer op og behandle dem samlet i main loopet. State-modellen gør det derfor tydeligt, at data først behandles, når systemet er klar.

Dette giver en bedre opdeling mellem modtagelse af data (MQTT) og behandling af data (main loop).

Til afsendelse af beskeder anvendes en Queue:

```
@publish_queue = Queue.new
```

Queue bruges til at samle beskeder, som skal sendes til MQTT. Dette gør det muligt at adskille logikken for at generere beskeder fra selve afsendelsen.

```
def enqueue_publish(topic:, payload:)
    logger.info("Queueing MQTT publish to '#{topic}'")
    publish_queue << {
      topic: topic,
      payload: payload
    }
  end
```

Publisher-threaden henter derefter beskeder fra køen:

```
  def flush_publish_queue(client, retain=true, qos=1)
```

```
    loop do
      message = publish_queue.pop(true)

      logger.info("Publishing MQTT message to '#{message[:topic]}':
#{message[:payload]}")
      client.publish(message[:topic], message[:payload], retain: retain, qos:
qos)
    end
  rescue ThreadError
    nil
  rescue => e
    logger.error("Error while flushing MQTT publish queue:
#{e.message}\n#{e.backtrace.join("\n")}")
  end
```

Queue-klassen i Ruby er thread-safe, hvilket betyder, at flere threads sikkert kan tilføje og hente beskeder uden ekstra synkronisering.

Vi har valgt at anvende både en state og en queue:

- **State** bruges til at samle ændringer fra MQTT, som main loopet senere behandler
- **Queue** bruges til at håndtere afsendelse af beskeder

Denne opdeling gør systemet mere overskueligt og adskiller ansvar mellem indgående og udgående data.

Kendt problem

Vi oplevede, at hvis forbindelsen til MQTT-serveren mistes, kan publisheren stoppe med at loope korrekt. Derfor anvender vi en non-blocking pop(true), så threaden ikke låser, men i stedet kan håndtere fejl og reconnecte.

Vi har også en PageHandler, som er en model, der holder styr på alle sider, der skal pinges.

Den har ansvar for at håndtere alle sider i systemet, herunder at tilføje, opdatere og fjerne sider, når der kommer ændringer fra MQTT.

En vigtig funktion i PageHandler er, at den kan gruppere sider, så de kun

pinges én gang, selv hvis flere sider refererer til samme URL:

```
  def uniq_pages_needing_ping
    logger.info("Calculating unique pages needing ping...")
    @uniq_pages_needing_ping ||= begin
      unique_pages =
pages.find_all(&:ready_for_ping?).group_by(&:normalize_path).map { |path,
pages| UniqPageNeedingPing.new(path, pages) }
      unique_pages.empty? ? nil : unique_pages
    end
```

Denne metode grupperer sider baseret på deres normaliserede URL (normalize_path). Det betyder, at hvis flere brugere har tilføjet den samme hjemmeside, vil den kun blive pinget én gang.

Resultatet er, at vi undgår unødvendige HTTP-requests. For eksempel, hvis 100 brugere har tilføjet den samme side, vil systemet kun ping'e den én gang og genbruge resultatet til alle.

Service

Vi har både en FaradayService og en CurlService. Begge bruges til at lave requests, men vi har valgt at dele ansvaret op mellem dem, da de er gode til forskellige ting.

Faraday håndterer de "rigtige" requests til backend og andre services, for eksempel når vi skal hente alle sider ved opstart, hente et bearer token eller hente favicons. Her er Faraday en fordel, fordi biblioteket indeholder en masse funktionalitet, som gør det nemmere at arbejde med almindelige API-kald, f.eks. redirects, retries og strukturering af requests og responses.

CurlService bruges derimod til selve ping-delen, hvor vi ønsker så rå og realistiske data som muligt. Her er formålet ikke bare at få et svar fra en side, men også at måle hvor lang tid de forskellige dele af requesten tager.

Vi bruger blandt andet denne linje til at fortælle curl, hvilke data vi vil have retur:

```
def curl_write_ms_response
    '-L -w
"status:%{http_code}\ndns_lookup:%{time_namelookup}\nconnect_to_page:
%{time_connect}\ntls_hand_shake:%{time_appconnect}\ntime_to_first_byt
e:%{time_starttransfer}\ntotal_time:%{time_total}\n"
--connect-timeout 5 --max-time 10'
  end
```

Ved at bruge curl får vi målinger direkte fra værktøjet selv, uden ekstra lag fra et Ruby-bibliotek. Det gør dataene mere realistiske i forhold til den faktiske svartid fra hjemmesiden.

Vi har derfor valgt at dele det op, fordi curl giver mere præcise og realistiske målinger til ping-delen, mens Faraday er bedre egnet til almindelige API-requests, hvor det er en fordel med mere indbygget funktionalitet.

Når main event loopet rammer, at der er sider, der skal pinges, kaldes følgende metode:

```
def run_curl_on_pages!
  max_threads = ENV.fetch("MAX_THREADS", 5).to_i
  queue = Queue.new

  page_handler.uniq_pages_needing_ping.each do |uniq_page|
    queue << uniq_page
  end

  threads = max_threads.times.map do
    Thread.new do
      loop do
          begin
            uniq_page = queue.pop(true)
```

```
          rescue ThreadError
            break
          end
        begin
          logger.info("Pinging #{uniq_page.path} for #{uniq_page.pages.size}
page(s)")
          response = CurlService.get_hashed_response(uniq_page.path, logger)
          uniq_page.response = response
          logger.info("Received response for #{uniq_page.path}: #{response}")
        rescue => e
          logger.error("Failed to ping #{uniq_page.path}: #{e.class} -
#{e.message}")
        end
      end
    end
  end

  threads.each(&:join)

  report_ping_response_to_mqtt!
  page_handler.uniq_pages_needing_ping = nil
rescue => e
  logger.error("Failed to ping pages: #{e.message}\n#{e.backtrace.join("\n")}")
end
end
```

Her anvender vi både threads og en queue til at håndtere ping af sider.

Vi bruger threads, fordi tidligere tests viste, at det kan tage lang tid at ping'e sider én ad gangen. Da arbejdet primært består af netværkskald, hvor programmet venter på svar, giver det mening at have flere requests i gang samtidig.

Ved at starte flere threads kan vi sende flere requests parallelt, hvilket reducerer den samlede ventetid.

Vi anvender en queue til at fordele arbejdet mellem threads.

Hvis vi i stedet brugte et almindeligt array, ville flere threads kunne forsøge at arbejde på de samme elementer samtidig, hvilket kunne føre til uforudsigelig adfærd.

Queue-klassen er thread-safe og fungerer ved, at hver thread henter en opgave med .pop. Når en thread tager en opgave fra queue'en, fjernes den, hvilket sikrer, at ingen andre threads behandler den samme side.

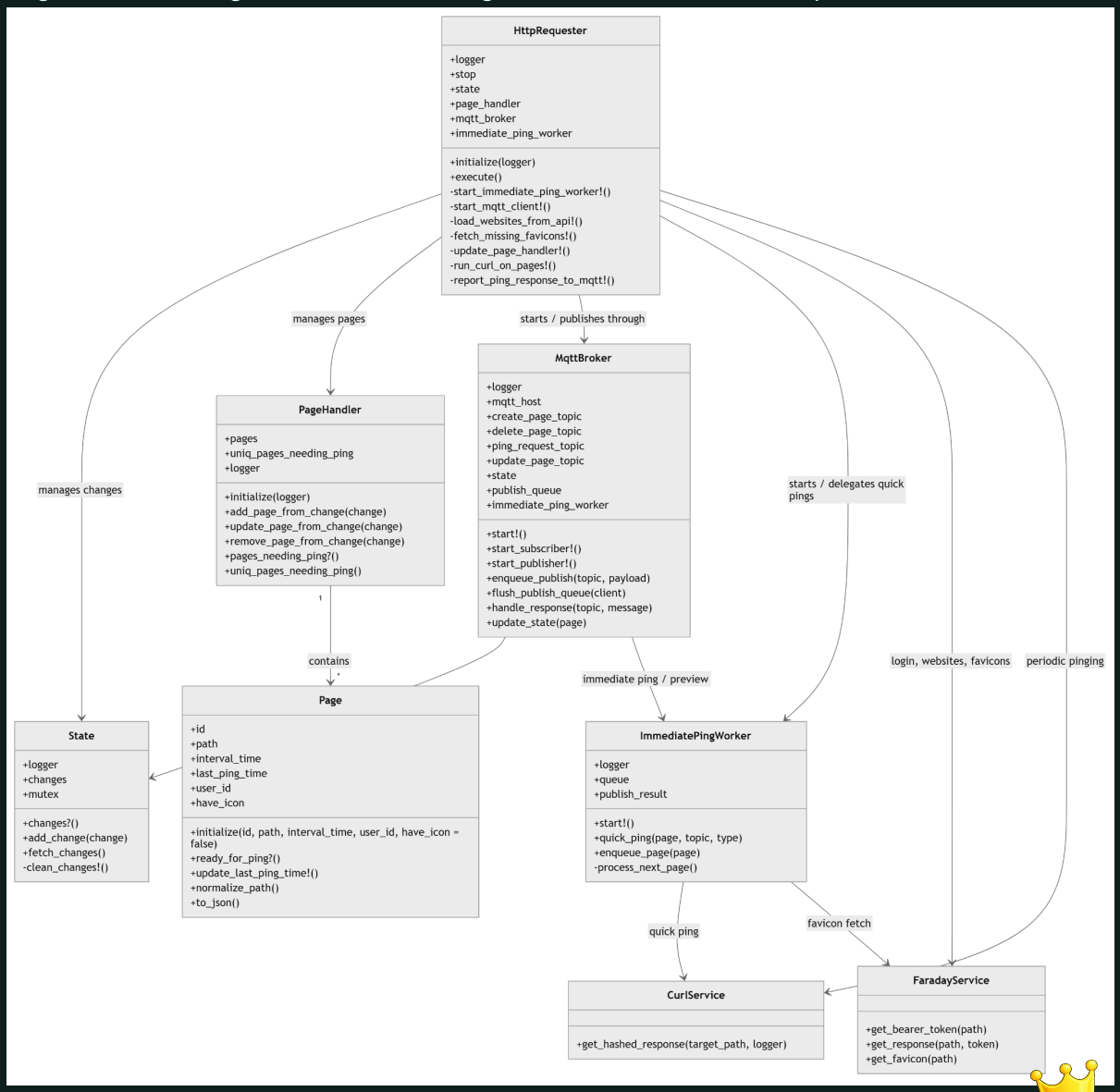
Threads.each(&:join) gør bare at vi venter på at alle threads er færdig

så sluter vi af med at køre

```
report_ping_response_to_mqtt!
```

som sætter igang med at sende beskeder til mqtt brokker.

Figur X: Oversigt over klasser og relationer i HTTP Requester



Diagrammet viser de centrale klasser i HTTP Requester samt deres relationer. HttpRequester fungerer som den overordnede styringsklasse, som koordinerer de øvrige komponenter. MqttBroker håndterer kommunikation med message brokeren, mens PageHandler og State styrer henholdsvis sider og indgående ændringer. ImmediatePingWorker håndterer hurtige ping-requests, og serviceklasserne bruges til netværkskommunikation.

Tests

Vi anvender **RSpec** til at teste vores applikation. RSpec er et testframework til Ruby, som er kendt for sin læsbarhed. Testene er skrevet på en måde, der gør det nemt at forstå, hvad der bliver testet, selvom syntaksen kan være lidt anderledes i starten.

Vi har valgt RSpec, da det gør det nemt at identificere fejl, især når tests køres automatisk i vores pipeline. Når en test fejler, er det tydeligt, hvad der er gået galt.

Vores tests kører automatisk hver gang der pushes kode til GitHub eller laves en pull request. Dette sikrer, at fejl opdages tidligt i udviklingsprocessen.

Dette håndteres via et GitHub Actions workflow:

```yaml
name: RSpec

on:
  push:
  pull_request:

jobs:
  rspec:
    runs-on: ubuntu-latest

    steps:
```

```yaml
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Build Docker image
        working-directory: ./http_requester
        run: docker build -t http_requester_test .

      - name: Run RSpec in Docker
        working-directory: ./http_requester
        run: docker run --rm http_requester_test bundle exec rspec
```

Workflowet fungerer ved, at der ved hvert push eller pull request startes en ny job på en Ubuntu-maskine. Her bliver koden hentet, Docker-imaget bygget, og derefter køres RSpec-testene inde i containeren.

Dette sikrer, at tests altid køres i et ensartet miljø.

Vores Docker image er bevidst holdt simpelt:

```dockerfile
FROM ruby:3.3

WORKDIR /app

RUN apt-get update && apt-get install -y curl ca-certificates tzdata

COPY Gemfile Gemfile.lock ./

RUN bundle install

COPY . .

CMD ["ruby", "main.rb"]
```

Vi tager udgangspunkt i et officielt Ruby-image og sætter arbejdsmappen til /app. Herefter installeres nødvendige systempakker:

- **curl** → bruges til HTTP og HTTPS-requests
- **ca-certificates** → nødvendigt for HTTPS requests
- **tzdata** → bruges til korrekt håndtering af tid og tidszoner, da vi bruger Time.now er dette vigtigt

Derefter kopieres vores dependencies (Gemfile) ind, og gems installeres. Til sidst kopieres resten af applikationen, og programmet startes med ruby main.rb.

Mangler og videre arbejde

I projektet var det planlagt at udvikle et sideløbende program, som skulle overvåge logs og give et visuelt overblik over fejl og systemets tilstand. Dette blev desværre ikke implementeret inden for projektets tidsramme.

Derudover er testdækningen begrænset, da der kun er udviklet unit tests for to klasser. For at opnå en mere robust løsning burde der være lavet tests for en større del af systemet.

Det var også planen at implementere automatiseret deployment, blandt andet ved brug af Rake tasks, så applikationen nemt kunne deployes. Dette blev dog prioriteret fra på grund af tid.

Konklusion

I dette projekt har vi udviklet et webbaseret monitoreringssystem, hvor brugere kan tilføje hjemmesider og få data omkring deres oppetid og svartider. Systemet er bygget op som en microservice-arkitektur med en klar opdeling mellem frontend, backend og en IoT-enhed.

Raspberry Pi'en fungerer som en worker, der løbende laver HTTP-requests på de valgte hjemmesider og sender resultaterne videre til backend via MQTT. Backend håndterer dataene og gemmer dem i databasen, hvorefter de vises på frontend i form af tabeller og grafer.

Gennem projektet har vi arbejdet med flere forskellige teknologier, herunder React, Go, .NET, Ruby, Docker, PostgreSQL og MQTT. Dette har givet os en god forståelse for, hvordan man opbygger et system med flere komponenter, der skal arbejde sammen.

Vi har opnået de vigtigste mål i projektet og fået en fungerende løsning, men

der er stadig områder, der kan forbedres. Især testdækning, log-overvågning og automatiseret deployment kunne gøre systemet mere robust.

Samlet set har projektet givet os erfaring med både udvikling, opsætning og drift af et system, og vi har fået en bedre forståelse for, hvordan forskellige teknologier kan spille sammen i en samlet løsning.